2026 오픈소스 컨트리뷰션 아카데미

Fedify 온보딩 세션

Fediverse, Fedify의 역할, 그리고 주변 생태계

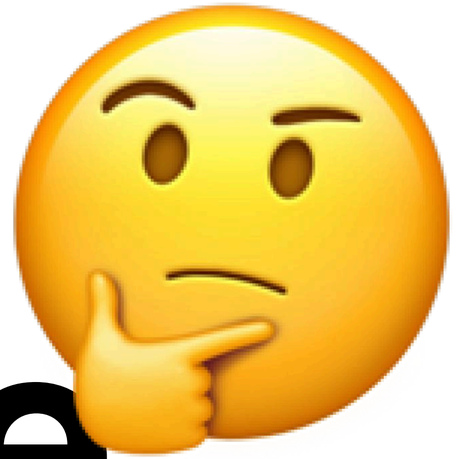


About Fediverse

Fedi + Verse

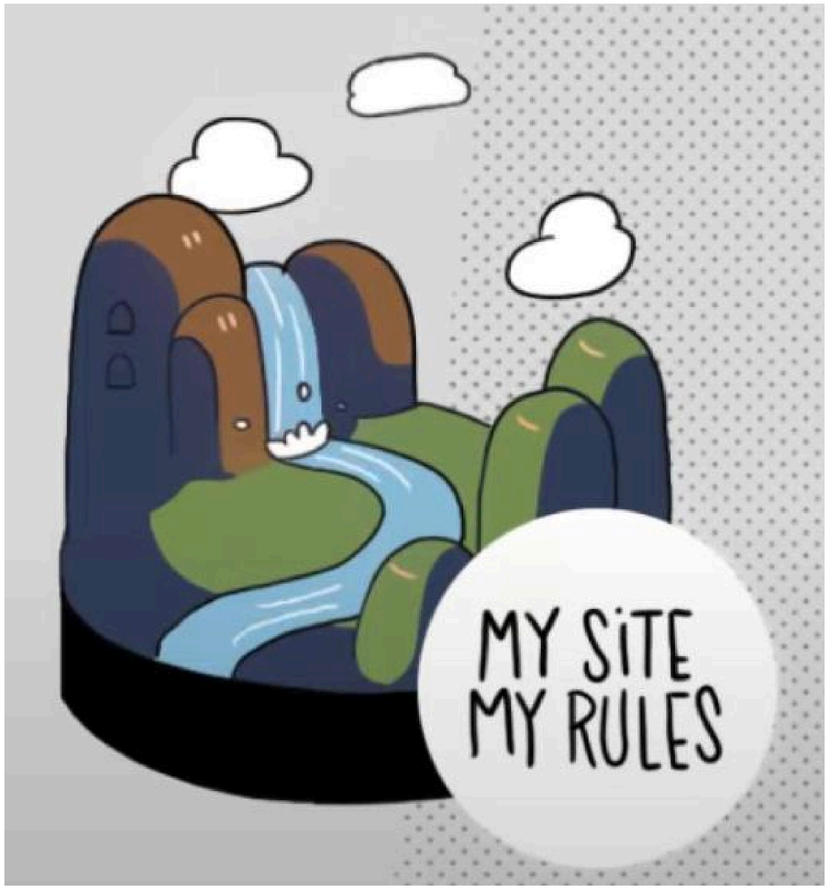
Fed(eration) + (Un)**iverse**

Fedi + Verse



Fed(eration) + (Un)**iverse**

하나의 우주





하나의 우주

하나의 서비스를 운영하는 서버

서비스에 가입이 되어있는 사람들

서비스를 구성하는 규칙

서로 다른 환경의 우주



어떤 서비스는 **fosstodon.org** 도메인에서 서빙되고

어떤 서비스는 **functional.cafe** 도메인에서 서빙되고

어떤 서비스는 **uri.life** 도메인에서 서빙되고 있다.



어떤 서비스는 **fosstodon.org** 도메인에서 서빙되고

어떤 서비스는 **functional.cafe** 도메인에서 서빙되고

어떤 서비스는 **uri.life** 도메인에서 서빙되고 있다.

이러한 서비스들끼리
어떻게 교류할 수 있을까?

어떤 사람은 **@gmail.com**을 쓰고

어떤 사람은 **@naver.com**을 쓰고

어떤 사람은 **회사 이메일**을 쓴다.

운영하는 회사도 다르고

서버의 위치도 다르고

내부 구현도 다르다.

그래도 메일은 서로 오간다.

SMTP라는

공통된 **메일 프로토콜**이 있기 때문이다.

ActivityPub도

서로 다른 소셜 서비스가

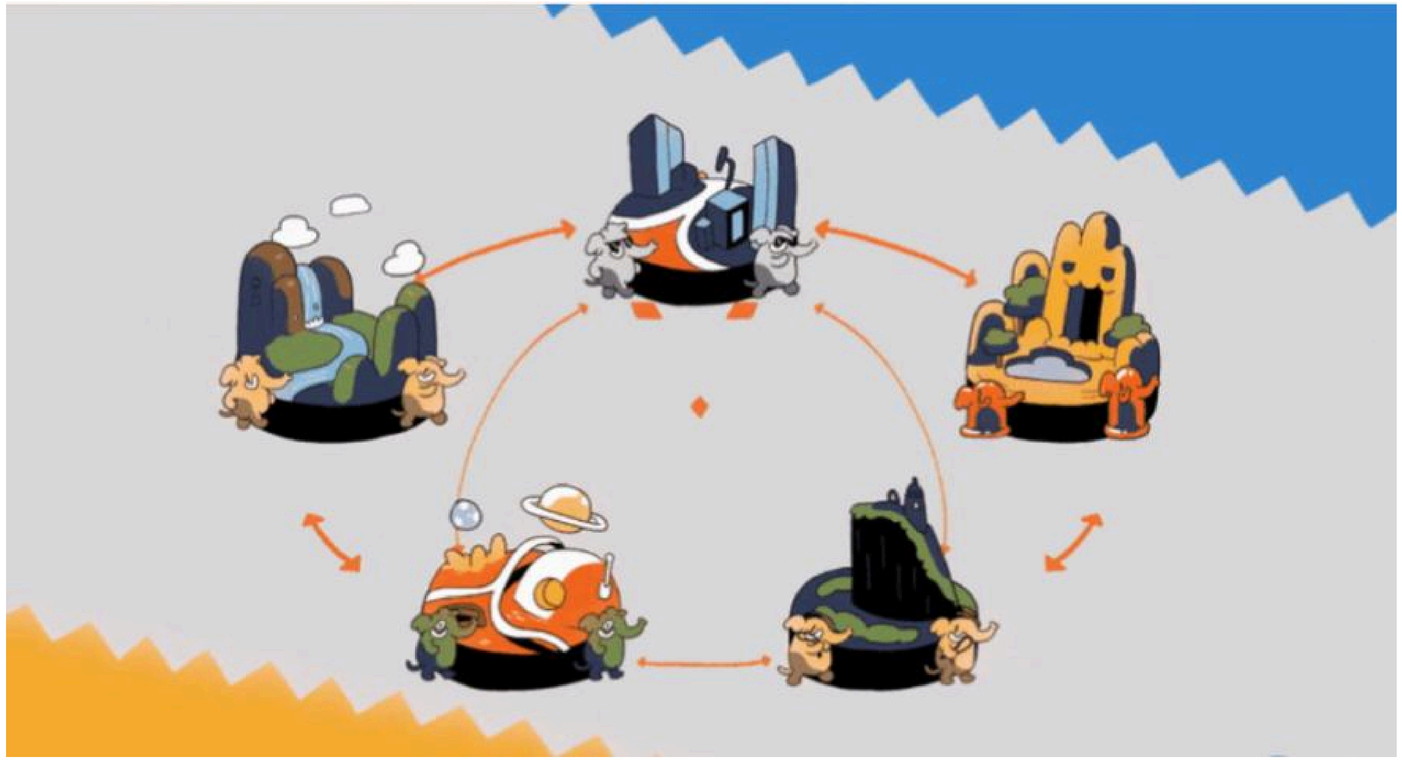
교류하기 위한 **공통된 프로토콜**이다.

ActivityPub이라는

공통된 프로토콜을 통해서 교류하기로

연합이 되었기에

“연합우주"



껍데기가 **Twitter** 같이 생겨도

껍데기가 **Reddit** 같이 생겨도

껍데기가 **Instagram** 같이 생겨도

다른 서비스를 사용하는 주민들끼리도

문제없이 소통이 가능하다.

ActivityPub은

탈중앙화된 서로 다른 소셜 서비스끼리

공통된 언어로 소통하여

하나의 타임라인에 있는 것처럼 해주는 프로토콜

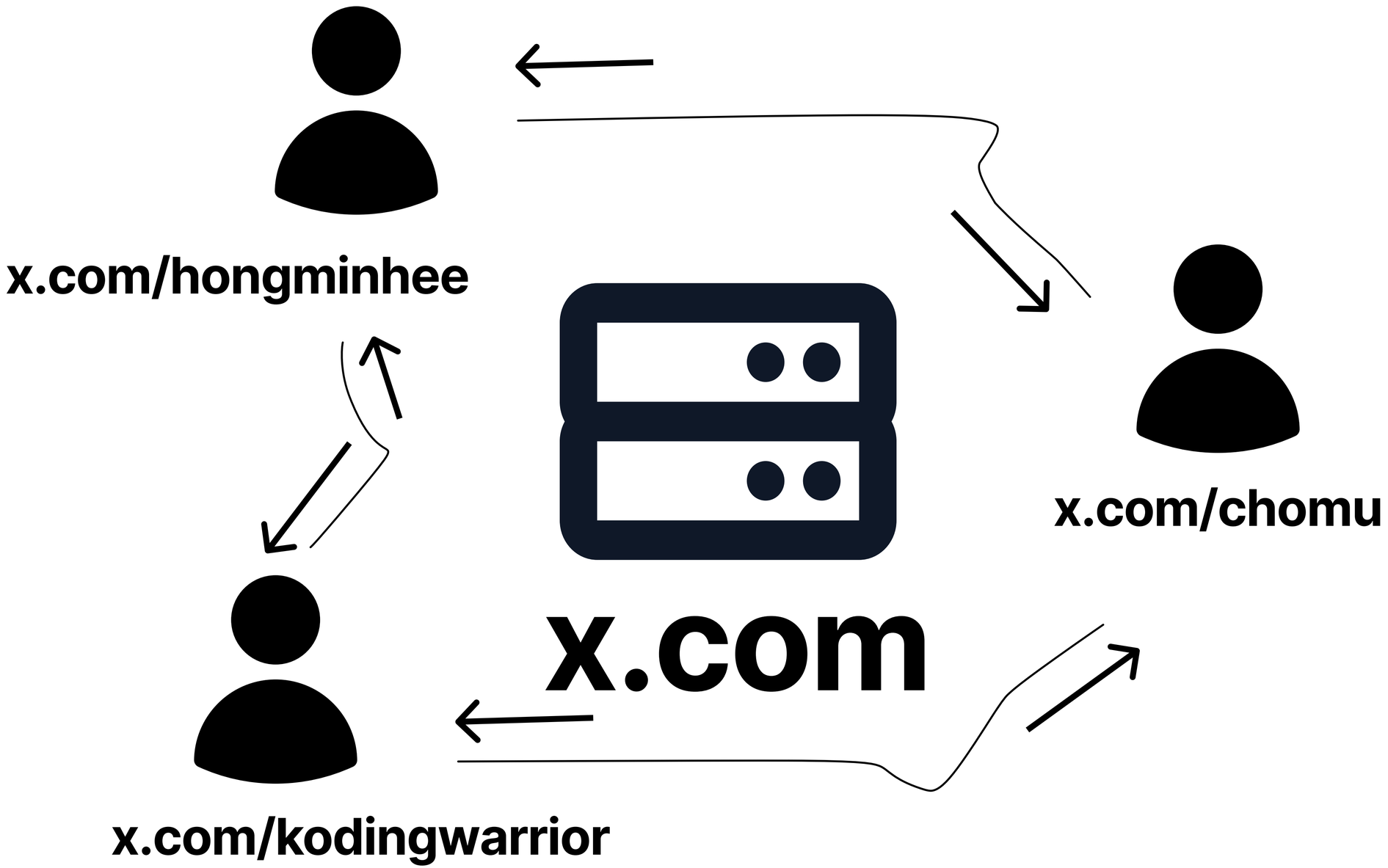
근데, 어쩌다
이런 기술이
나오게 되었나요?

대부분의 소셜 서비스는

한 운영 주체에 의해서 운영됩니다

(중앙화된 운영)

Twitter(현 X) / Instagram / Threads


x.com/hongminhee
x.com/chomu
x.com
x.com/kodingwarrior

일부 서비스들은

내 의지와 상관없이

중단되기도 합니다

카카오스토리 / 미투데이 / 싸이월드 / Google+

내 데이터(친구관계, 글)은 안전한가?

내가 쓴 글이 **불합리한 기준으로 검열**되면 어떡하지?

내 피드는 안전한가?

이상한 알고리즘이 들어가면 어떡하지?

내 의지와 상관없이 서버가 종료되면 어떡하지?

내가 고르는 **안전한 공간** (혹은 타임라인)

내가 믿을 수 있는 운영 주체 (혹은 나 자신이 될 수도..)

내 취향에 맞는 서비스 (사진 중심, 텍스트 중심, ...)

내 데이터를 다루는 방식의 투명성 보장

내가 고르는 **안전한 공간** (혹은 타임라인)

내가 믿을 수 있는 운영 주체 (혹은 나 자신이 될 수도..)

내 취향에 맞는 서비스 (사진 중심, 텍스트 중심, ...)

내 데이터를 다루는 방식이 투명성 보장

그러면서,

서로 다른 서비스 쓰고 있어도 (탈중앙화)

같은 타임라인에 있는 것과 같은

이상향을 만들어 볼 수는 없을까?

ActivityPub

투명성 있고 **안전**하고

상호호환이 되는 웹을 만들기 위한

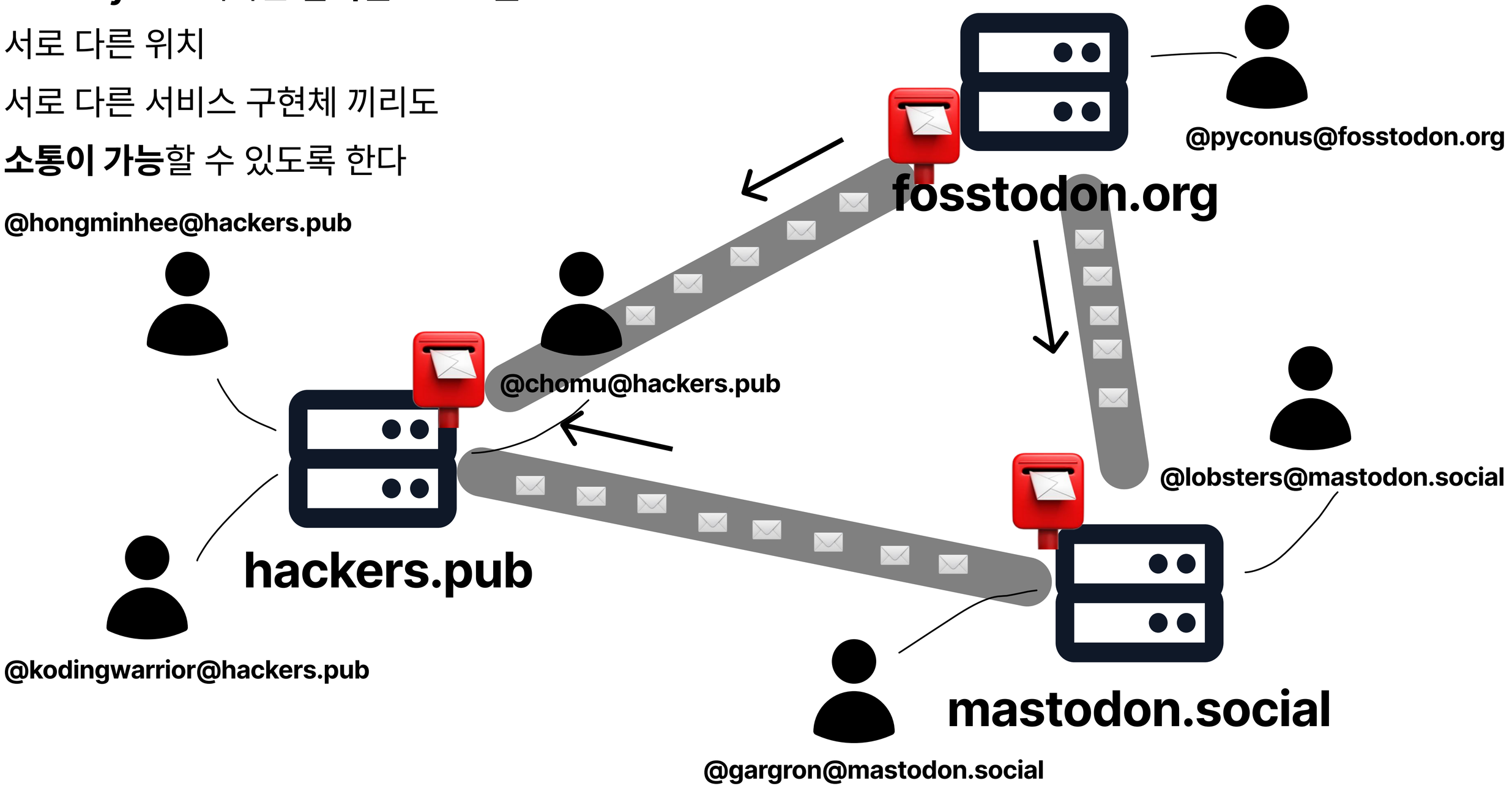
W3C 표준

ActivityPub 이라는 **합의된 프로토콜**로

서로 다른 위치

서로 다른 서비스 구현체 끼리도

소통이 가능할 수 있도록 한다



서비스의 내부 구현이

Ruby/Java/Python/Javascript/...

어떻게 구현되어 있든 상관없다

인터페이스만 맞으면 된다

Fediverse **E**nhancement **P**roposal

“연합우주를 더 풍성하게 만들자!” 합의된 기준

물물거래(FEP-0837)도 할 수 있게 하자!

친구추천(FEP-7aa9)도 할 수 있게 하자!

마크다운(FEP-c16b)도 지원해보자!

그렇다면,

Fedify란 무엇인가요?

ActivityPub 기반의

서비스를 만들기 위해

웹 서버 프레임워크의 기능을

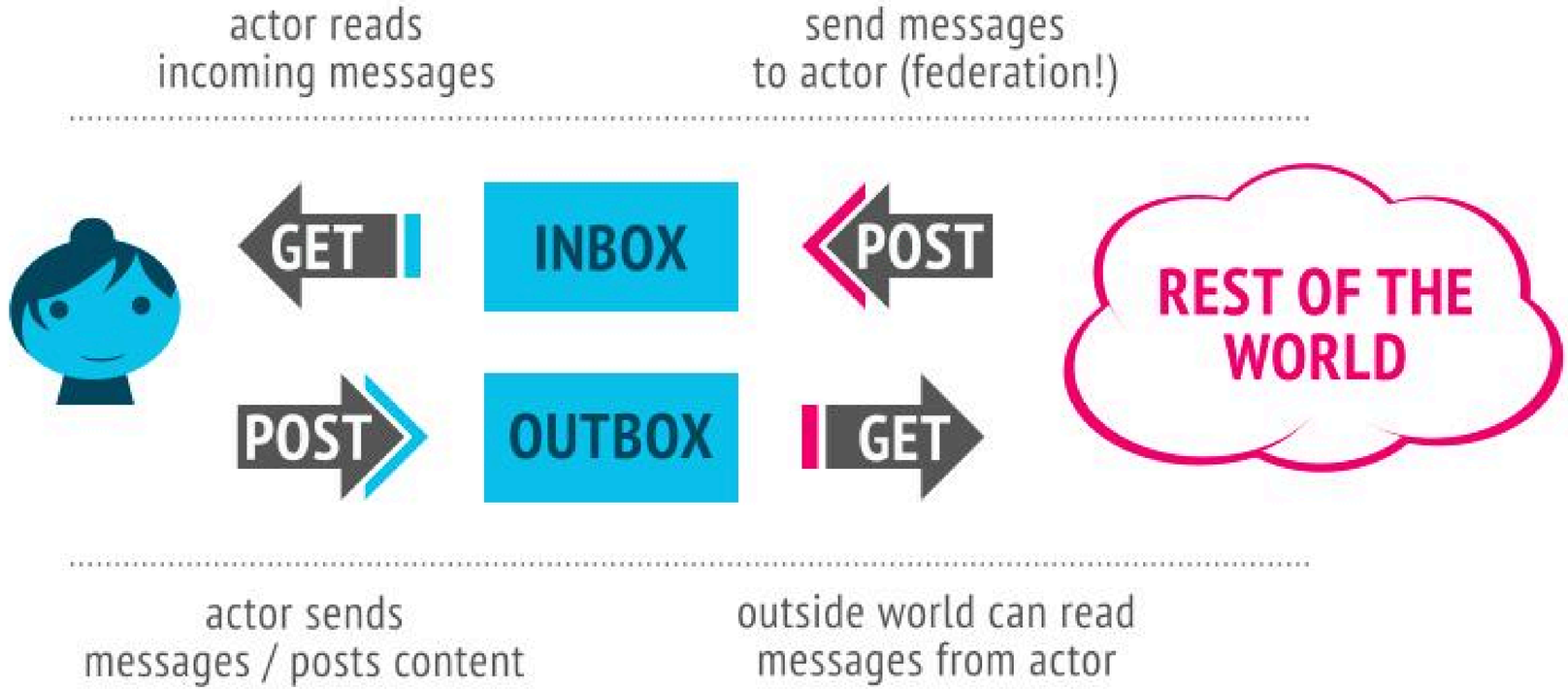
확장해주는 유틸리티 라이브러리



기술적인 내용이 언급될 수 있습니다

Mental model

핵심 키워드 : **Actor, Activity, Object, Inbox, Outbox**



Mental model

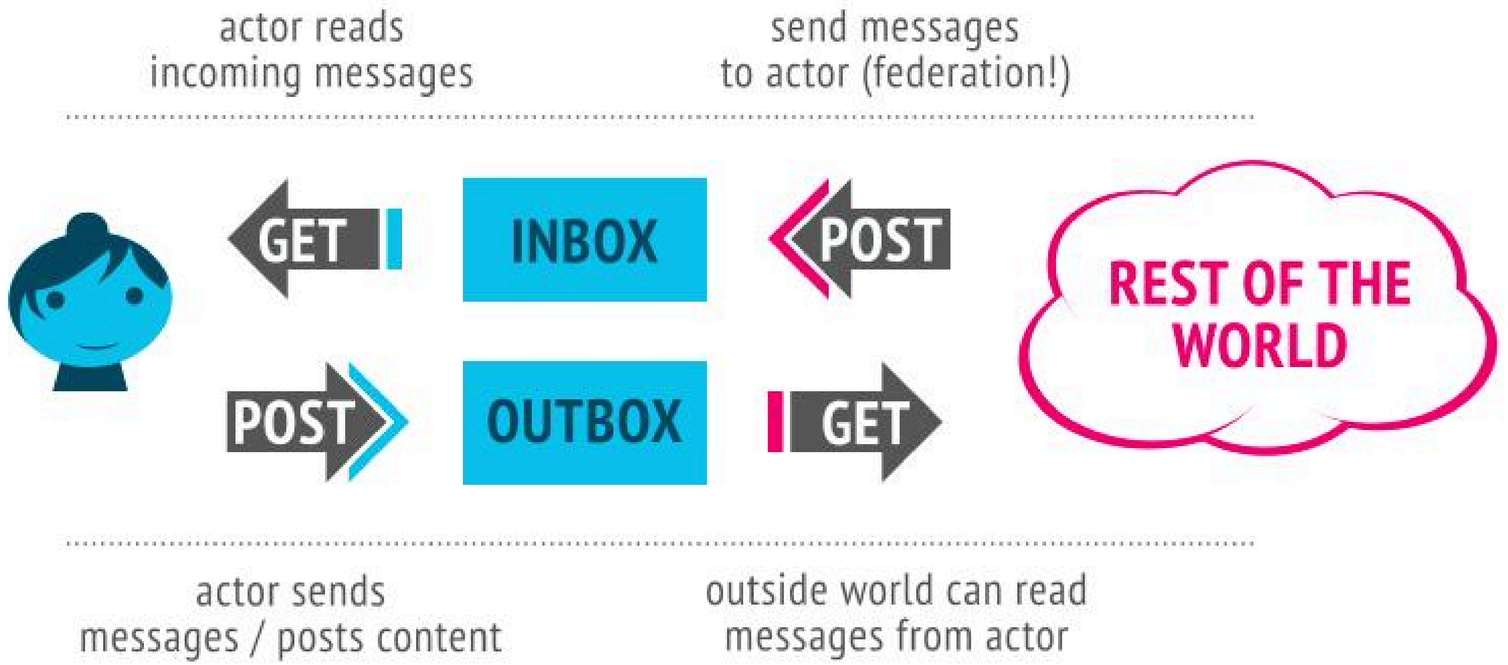
핵심 키워드 : **Actor**, Activity, Object, Inbox, Outbox

Actor : 연합우주를 구성하는 사용자를 지칭

사용자

혹은 조직

혹은 서버 그 자체



Mental model

핵심 키워드 : Actor, **Activity**, Object, Inbox, Outbox

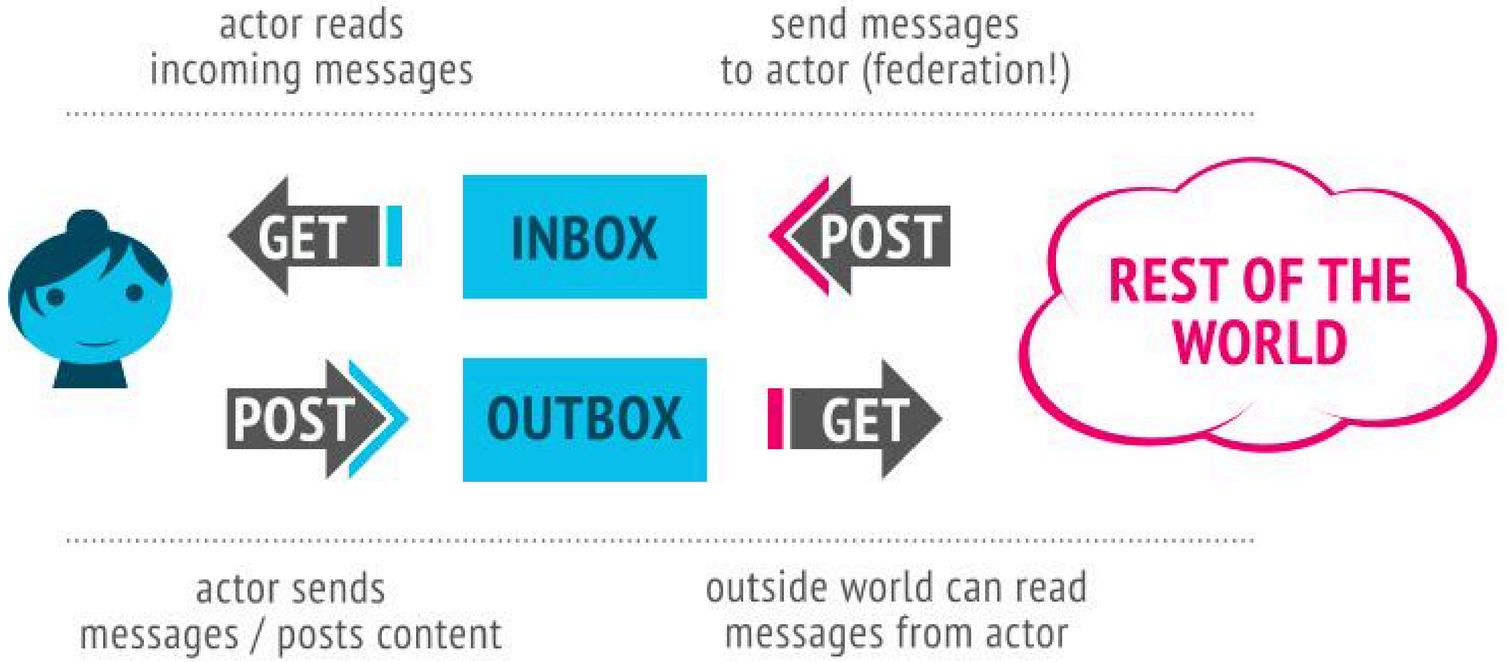
Activity : 사용자의 활동

Create(Follow)

Create(Note)

Create(Event)

Like(Note)



Mental model

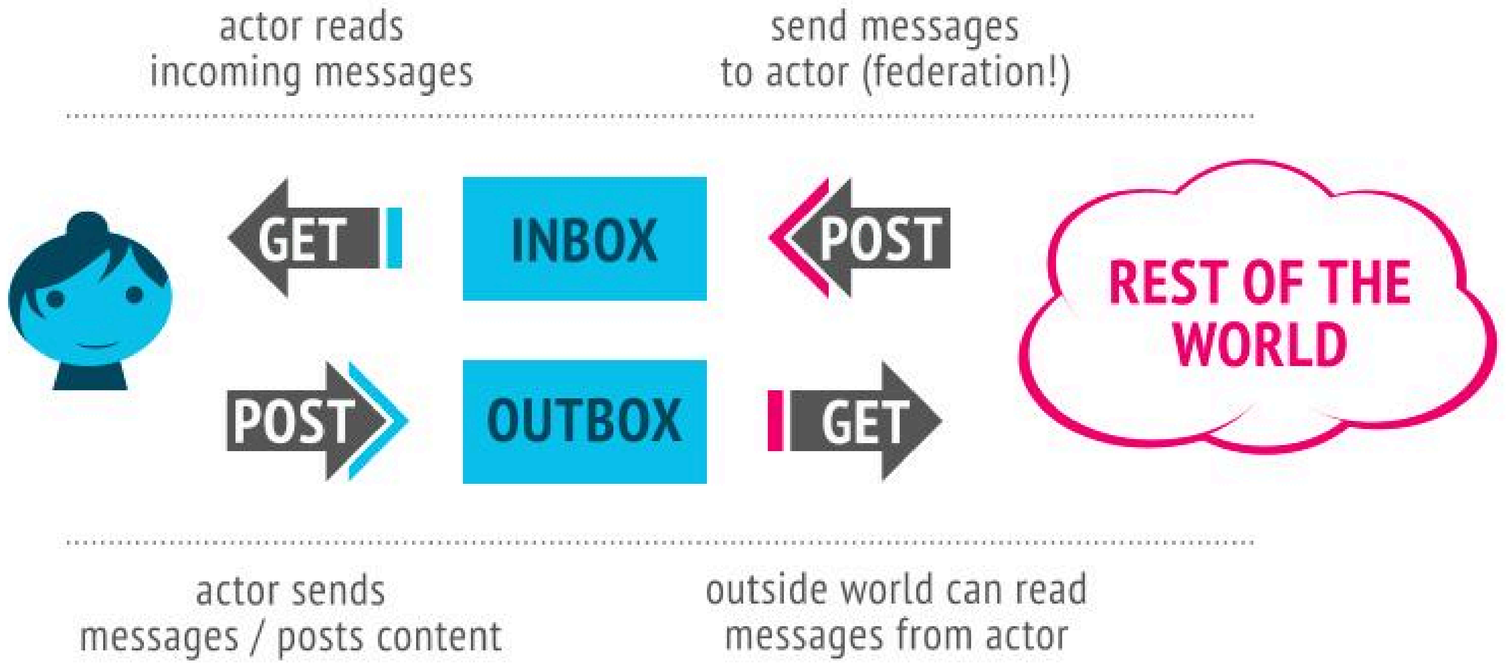
핵심 키워드 : Actor, Activity, **Object**, Inbox, Outbox

Object : ActivityPub 서버가 가지고 있는 리소스

Note, Event, Follow

Person, Service

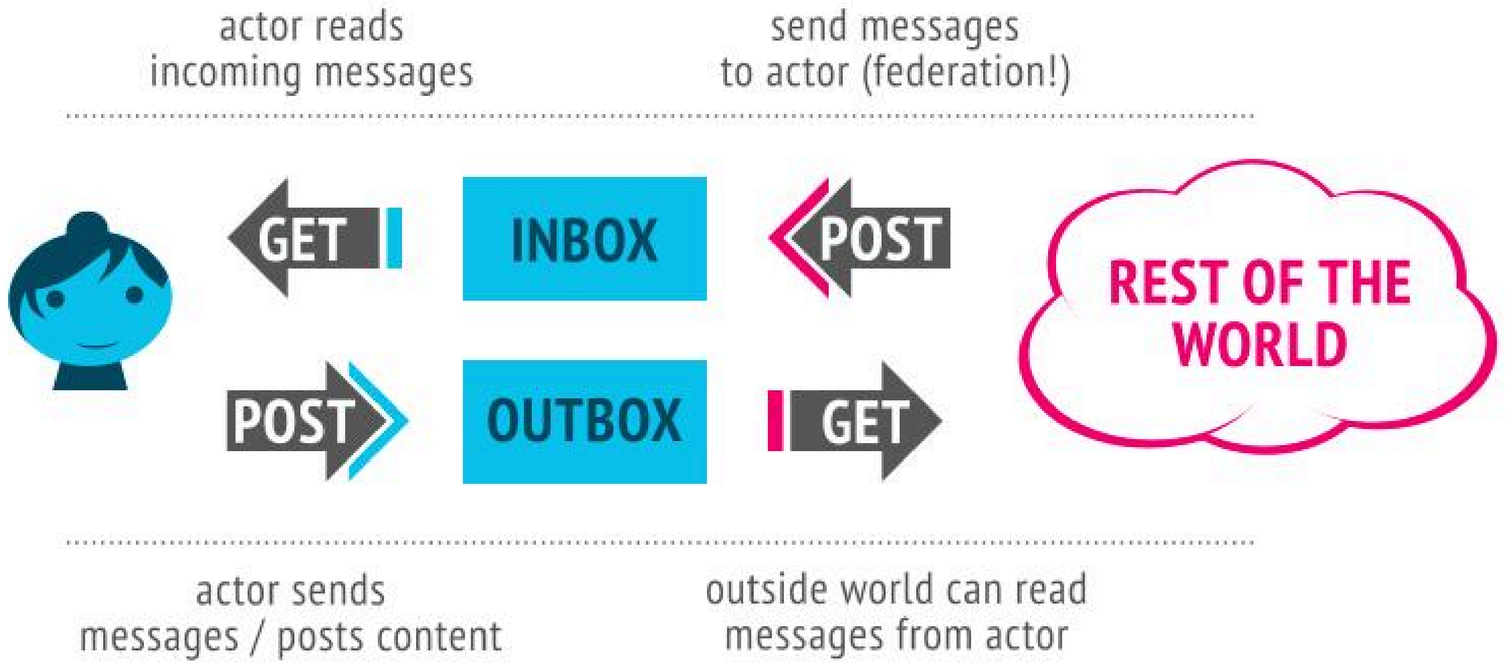
Place, Image



Mental model

핵심 키워드 : Actor, Activity, Object, Inbox, **Outbox**

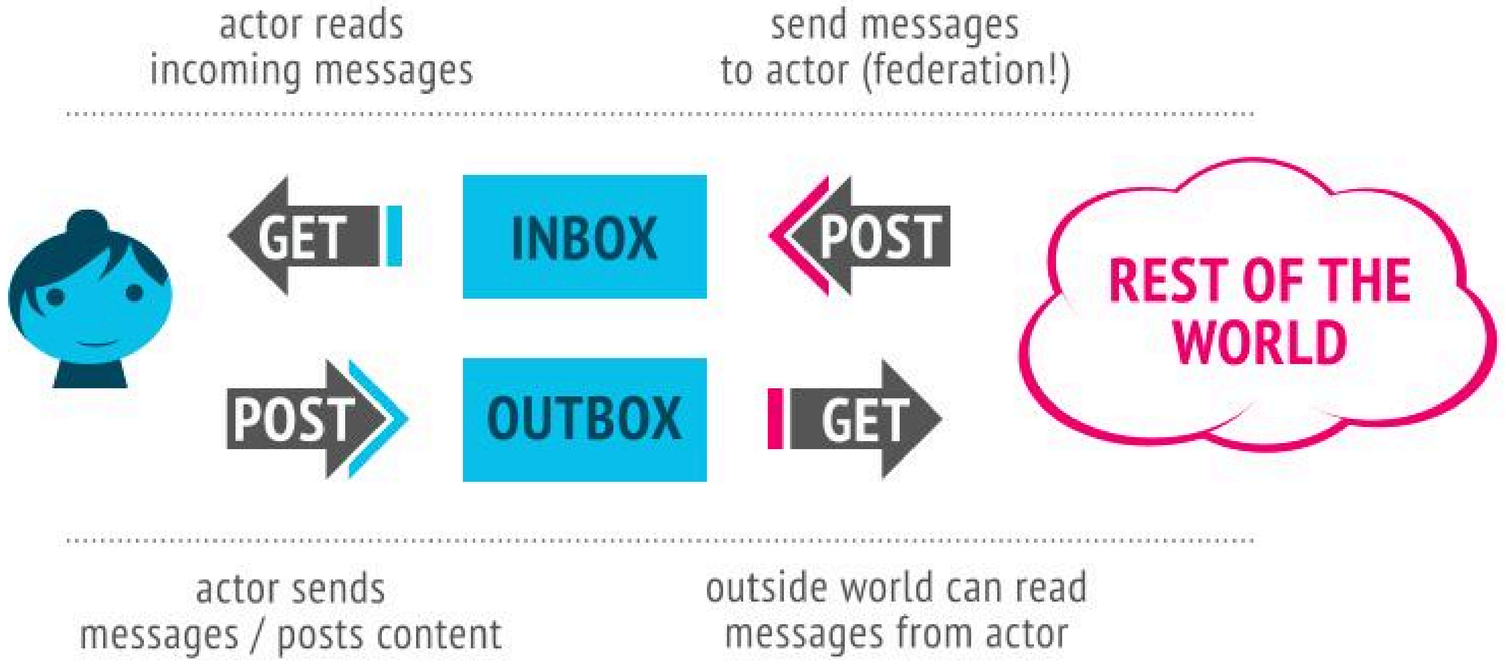
Outbox : 사용자의 ActivityPub이 외부로 나가는 통로



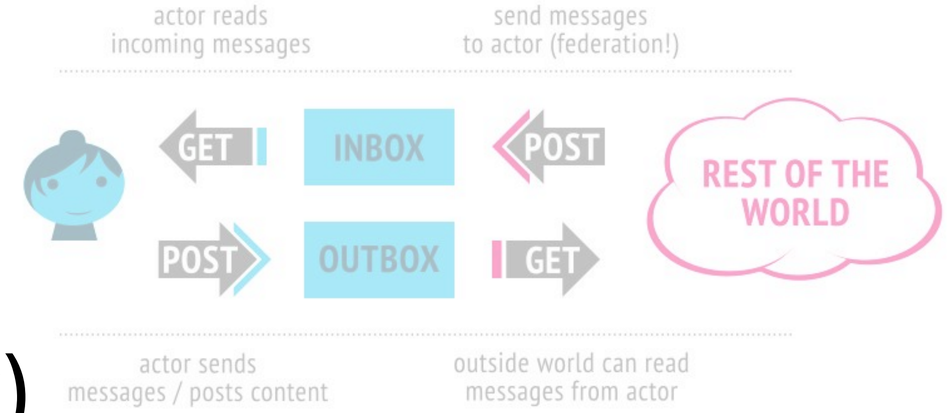
Mental model

핵심 키워드 : Actor, Activity, Object, **Inbox**, Outbox

Inbox : 외부에서 흘러들어온 ActivityPub 메시지가 처리되는 곳



근데, 어떻게 구현하지?



메시지가 쌓이는 큐 (Inbox, Outbox)

올바른 접근 제어(친구만 공개, ...)를 위한 **서명 검증**

webfinger를 통한 **리소스 조회**

“application/ld+json” Content-Type에 대한 처리

Inbox로 들어오는 Activity에 대한 올바른 처리

Fedify의 역할은?

ActivityPub 서비스를 개발하는데 특화된 도구를 제공한다

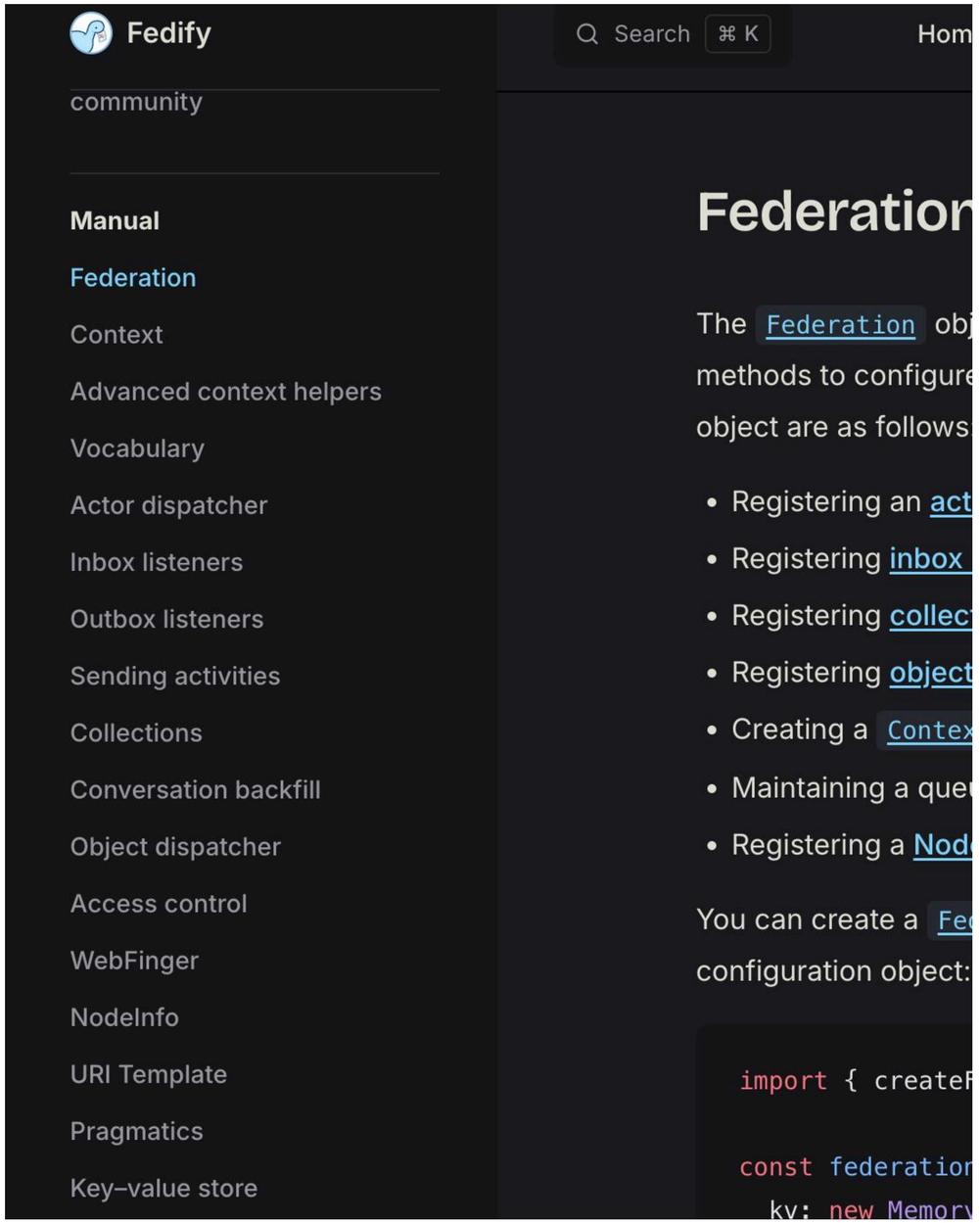
그리고, 도구를 사용하는 잘 만들어진 문서를 제공한다.

ActivityPub 소프트웨어는

어떻게 구성이 되는가?

어떤 기능의 구현을 위해서

도구를 **어떻게 응용**하는가



Actor에 대한 정보를
어떻게 접근할 것인가 (Key Pair 포함)

```
(parameter) federation: Federation<unknown>
federation
  .setActorDispatcher(…)
  .mapHandle(async (ctx, username) ⇒ {…})
  .mapAlias((ctx, resource: URL) ⇒ {…})
  .setKeyPairsDispatcher(this.handleKeyPairs.bir

federation.setOutboxDispatcher(
  '/ap/actors/{identifier}/outbox',
  async (ctx, identifier) ⇒ {…},
);

federation
  .setInboxListeners('/ap/actors/{identifier}/ir
  .setSharedKeyDispatcher((ctx) ⇒ ({
    identifier: new URL(ctx.canonicalOrigin).hos
  }))
  .on(APFollow, async (ctx, follow) ⇒ {…})
  .on(Undo, async (ctx, undo) ⇒ {…})
  .on(Accept, async (ctx, accept) ⇒ {…})
  .on(Reject, async (ctx, reject) ⇒ {…})
  .on(Create, async (ctx, create) ⇒ {…})
  .on(Announce, async (ctx, announce) ⇒ {…});
```

Actor에서 바깥으로 나가는 Activity를 어떻게 표시할 것인가

```
(parameter) federation: Federation<unknown>
federation
  .setActorDispatcher(…)
  .mapHandle(async (ctx, username) => {…})
  .mapAlias((ctx, resource: URL) => {…})
  .setKeyPairsDispatcher(this.handleKeyPairs.bir
```

```
federation.setOutboxDispatcher(
  '/ap/actors/{identifier}/outbox',
  async (ctx, identifier) => {…},
);
```

```
federation
  .setInboxListeners('/ap/actors/{identifier}/ir
  .setSharedKeyDispatcher((ctx) => ({
    identifier: new URL(ctx.canonicalOrigin).hos
  }))
  .on(APFollow, async (ctx, follow) => {…})
  .on(Undo, async (ctx, undo) => {…})
  .on(Accept, async (ctx, accept) => {…})
  .on(Reject, async (ctx, reject) => {…})
  .on(Create, async (ctx, create) => {…})
  .on(Announce, async (ctx, announce) => {…});
```

Actor의 inbox로 들어오는

Activity를 어떻게 처리할 것인가?

* **Create**(Note) : **게시글 작성**
* **Reject**(Follow) : **팔로우 요청 거절**
* **Announce**(Note) : **게시글 퍼나르기**

```
(parameter) federation: Federation<unknown>
federation
  .setActorDispatcher(…)
  .mapHandle(async (ctx, username) ⇒ {…})
  .mapAlias((ctx, resource: URL) ⇒ {…})
  .setKeyPairsDispatcher(this.handleKeyPairs.bir

federation.setOutboxDispatcher(
  '/ap/actors/{identifier}/outbox',
  async (ctx, identifier) ⇒ {…},
);
```

```
federation
  .setInboxListeners('/ap/actors/{identifier}/ir
  .setSharedKeyDispatcher((ctx) ⇒ ({
    identifier: new URL(ctx.canonicalOrigin).hos
  }))
  .on(APFollow, async (ctx, follow) ⇒ {…})
  .on(Undo, async (ctx, undo) ⇒ {…})
  .on(Accept, async (ctx, accept) ⇒ {…})
  .on(Reject, async (ctx, reject) ⇒ {…})
  .on(Create, async (ctx, create) ⇒ {…})
  .on(Announce, async (ctx, announce) ⇒ {…});
```

Fedify의 역할은?

ActivityPub 서비스를 개발하는데 특화된 도구를 제공한다

그리고, 도구를 사용하는 잘 만들어진 문서를 제공한다.

최종적으로는, ActivityPub 서비스 개발의 진입장벽을 낮춘다.

그리고,
여러분의 역할

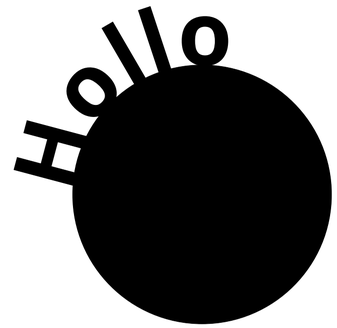
1. Fedify 에 직접 기여한다.
2. Fedify 자매 프로젝트에 기여한다.
3. 생태계를 넓히는 방향으로 기여한다.

Fedify의 자매 프로젝트

Hollo / BotKit / DrFed / Hackers’ Pub

Fedify의 자매 프로젝트

Hollo / BotKit / DrFed / Hackers' Pub



1인 저널리즘 혹은 블로깅 특화!

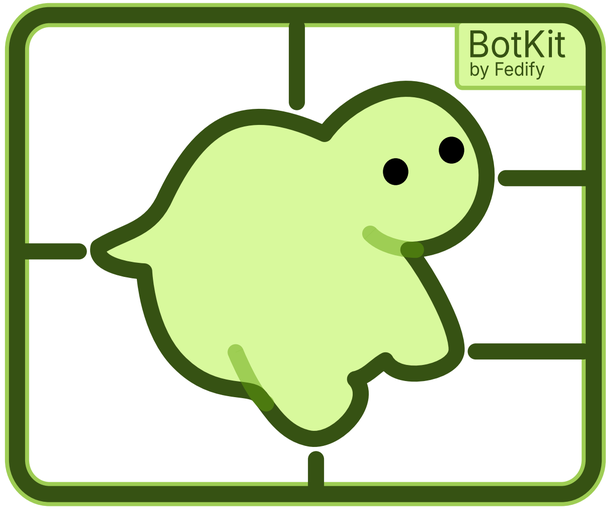
저사양에서 무난하게 돌아가는 Self-hosted 서비스

Fedify의 자매 프로젝트

Hollo / **BotKit** / DrFed / Hackers' Pub

연합우주에서 봇계정을 굴리기 위한

경량화된 프레임워크!



Fedify의 자매 프로젝트

Hollo / BotKit / **DrFed** / Hackers’ Pub



ActivityPub 서비스 개발을 위한

온라인 디버거 (**Activitypub.academy 대체재**)

NLNet 펀딩도 받고 있어요 (Coming soon…)

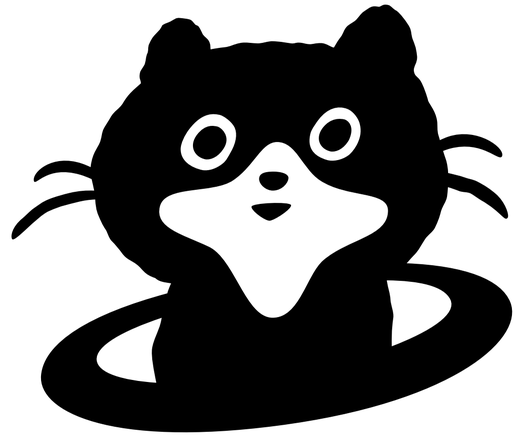
Fedify의 자매 프로젝트

Hollo / BotKit / DrFed / **Hackers' Pub**

Fedify의 Best Practice로

굉장히 밀어붙이고 있는 블로깅 플랫폼

Velog, Zenn.dev, dev.to 게섯거라



또한, 프레임워크는

사용처/생태계가 있어야 의미를 가진다

Hackers' Pub **(블로깅 플랫폼)**

Moim.live **(이벤트 개최 플랫폼)**

(그리고, 여기에 **여러분이 개발하는 프로덕트**를 입력)

OSSCA 프로그램은 3개의 트랙으로 진행

기초반 / 기본반 / 응용반

사실, 병행해서 진행하셔도 됩니다.

OSSCA 프로그램은 3개의 트랙으로 진행

기초반 / 기본반 / 응용반

튜토리얼을 진행하면서, **기본기를 익힌다.**

Twitter/Instagram/Reddit 등 다양한 예시가 준비되어 있음.

OSSCA 프로그램은 3개의 트랙으로 진행

기초반 / **기본반** / 응용반

Fedify 혹은 자매프로젝트에 기여한다.

Hollo / BotKit / DrFed / Hackers’ Pub

이번 OSSCA 프로그램은 3개의 트랙으로 진행

기초반 / 기본반 / **응용반**

Fedify를 응용한 다른 프로젝트를 만들어서 **생태계를 확장**한다.

아이디에이션 / 요구사항 분석 / 역할분담 / 본격적인 개발

Fedify

많은 관심 부탁드립니다

https://github.com/fedify-dev/fedify

여러분이 끌리는 자매프로젝트도

많은 관심 부탁드립니다

BotKit : **https://github.com/fedify-dev/botkit**

Hackers' Pub : **https://github.com/hackers-pub/hackerspub**

Hollo : **https://github.com/fedify-dev/hollo**

DrFed : **https://github.com/fedify-dev/drfed**

여러분은 **ActivityPub 생태계**

그것도 **동아시아를 중심**으로

ActivityPub 기반의

오픈 웹 생태계를

발전시키는 주역이 되는 것이다

함께 보면 좋은 자료

- https://www.w3.org/TR/activitypub/
- https://www.w3.org/TR/activitystreams-vocabulary/
- https://activitypub.rocks/
- https://slides.kodingwarrior.dev/fediverse-onboarding-ubucon2025.pdf

감사합니다